

Guía del proyecto — Ferretería Los Maestros (Entrega 1)

Hola Segas 🙌. Esta guía explica **qué hace cada pieza** del sitio y **cómo se conecta todo**, para que puedas replicarlo, entenderlo y defenderlo en la presentación. Está escrita paso a paso, estilo "lápiz y papel".

Idea clave que se repite en todo el proyecto: **el HTML pone las "cajas" en pantalla, el CSS las viste, y el JavaScript les da vida**. Los tres archivos se conectan mediante `id`, `class` y nombres de funciones. Si esos nombres calzan, todo funciona; si no calzan, algo queda "muerto". Por eso presta atención a los `id`.

1. Qué vamos a construir y con qué

La Entrega 1 es el **frontend** (lo que se ve en el navegador) de la tienda "Ferretería Los Maestros", más su **panel de administración**. Se usa:

- **HTML5** para la estructura (el esqueleto).
- **CSS externo** con **Bootstrap 5** (vía CDN) + una hoja propia, para el diseño responsive.
- **JavaScript** puro (sin librerías) para: mostrar productos, el carrito con `localStorage`, y validar formularios.
- **GitHub + GitHub Pages** para publicarlo.

Todavía **no** hay React ni base de datos: eso viene en entregas siguientes. Aquí los datos viven en arreglos de JavaScript.

2. Cómo se organiza y se conecta todo (¡el mapa!)

Esta es la parte que más importa para no perderse. El proyecto tiene esta forma:

```
ferreteria-los-maestros/
├─ *.html                (las páginas de la tienda)
├─ admin/*.html          (las páginas del administrador)
├─ css/estilos.css       (diseño de la tienda)
├─ css/admin.css         (diseño del panel admin)
└─ js/*.js               (toda la lógica)
```

Cómo se enlazan los archivos dentro de una página HTML:

- En el `<head>` cargamos el CSS con `<link href="css/estilos.css" rel="stylesheet">`.
- Al final del `<body>` cargamos los JS con `<script src="js/archivo.js"></script>`.

El orden de los `<script>` importa muchísimo. Un archivo solo puede usar lo que ya se cargó antes. Por eso el orden siempre es:

1. **Bootstrap** (para que funcionen el menú y los componentes).
2. **Los datos** (`datos-productos.js` , etc.) → definen las variables `PRODUCTOS` , `USUARIOS` , `REGIONES` .
3. **La lógica** (`carrito.js` , `tienda.js` , `validaciones.js` , ...) → usan esos datos.
4. `comun.js` al final → usa todo lo anterior (por ejemplo, el contador del carrito).

Si pusieras `tienda.js` **antes** de `datos-productos.js` , `tienda.js` intentaría usar `PRODUCTOS` cuando todavía no existe → error. Ese es exactamente el tipo de "conector faltante" que hay que cuidar.

Cómo se conectan HTML y JS: el HTML deja "espacios en blanco" con un `id` (por ejemplo `<div id="grid-productos"></div>`), y el JS los rellena buscándolos con `document.querySelector("#grid-productos")` . **El texto del `id` en el HTML tiene que ser idéntico al que busca el JS.**

Rutas relativas (tienda vs admin): las páginas del admin están en la carpeta `admin/` , así que para "subir" un nivel usan `../` . Por eso en el admin verás `../css/admin.css` y `../js/comun.js` , mientras que en la tienda es `css/estilos.css` y `js/comun.js` .

3. Cómo armar el proyecto desde cero

1. Crea una carpeta llamada `ferreteria-los-maestros` .
2. Dentro, crea las subcarpetas `css` , `js` e `img` , y una carpeta `admin` .
3. Ve creando los archivos `.html` , `.css` y `.js` (los tienes todos listos; cópialos tal cual).
4. Para probar, abre `index.html` con doble clic. ¡Listo, ya lo ves funcionando!

Tip: instala la extensión **Live Server** en VS Code. Te abre el sitio en el navegador y lo recarga solo cada vez que guardas. Cómodo para desarrollar.

4. El HTML pieza por pieza

Todas las páginas siguen la misma plantilla. Veamos las partes.

4.1 El encabezado `<head>`

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ferretería Los Maestros – Inicio</title>
  <link href="css/vendor/bootstrap.min.css" rel="stylesheet">
  <link href="css/vendor/bootstrap-icons.min.css" rel="stylesheet">
  <link href="css/estilos.css" rel="stylesheet">
</head>
```

- `charset="UTF-8"` : permite tildes y ñ sin que se vean raras.
- `viewport` : hace que el sitio se adapte al ancho del celular (responsive).
- `<title>` : el texto de la pestaña del navegador.
- Los `<link>` : cargan Bootstrap, los íconos y **nuestra** hoja de estilos. La nuestra va **última** para poder "ganarle" a Bootstrap cuando queremos otro color.

Importante — Bootstrap LOCAL, no por internet. Bootstrap está guardado dentro del proyecto en `css/vendor/` y `js/vendor/` (en vez de cargarlo desde un CDN de internet). ¿Por qué? Porque si la red o un VPN bloquea el CDN, el CSS no llega y el sitio se ve **todo descuadrado** (las tarjetas se apilan gigantes). Con la copia local funciona siempre, incluso sin internet — ideal para la presentación en sala. En las páginas del admin la ruta lleva `../` delante (`../css/vendor/...`) porque están en la subcarpeta `admin/`.

4.2 La barra de navegación (navbar)

Es HTML de Bootstrap. Lo importante:

- `class="navbar ... fixed-top"` : barra fija arriba.
- El botón `navbar-toggler` es la **hamburguesa** que aparece en el celular. Con `data-bs-target="#menu"` le dice a Bootstrap qué menú abrir/cerrar.
- Cada enlace `<a class="nav-link" href="productos.html">` navega a otra página (esto cumple el requisito de "hipervínculos para navegar entre páginas").
- `<span class="... contador-carrito">0</span>` : aquí el JS escribe cuántos productos hay en el carrito.

4.3 El contenido `<main>` y la semántica

Usamos etiquetas **semánticas** (que tienen significado, no solo `<div>`):

- `<main>` : el contenido principal.
- `<section>` : una sección temática.

- `<article>` : un contenido independiente (cada blog es un `<article>`).
- `<nav>` : navegación (la navbar y las "migas de pan").
- `<footer>` : el pie de página.
- `<h1>` , `<h2>` ...: títulos en orden de importancia (solo **un** `<h1>` por página).

Esto es parte de la rúbrica: "estructura clara y semántica con secciones, encabezados, párrafos y listas".

4.4 El pie de página `<footer>`

Contiene datos de la tienda, enlaces y un campo para el boletín. Se repite en las páginas para dar consistencia.

5. El CSS externo y responsive

Está en `css/estilos.css` (tienda) y `css/admin.css` (admin). Puntos clave que puedes explicar:

- **Variables de color** al inicio (`--naranja` , `--carbon` , ...). Se definen una vez y se usan con `var(--naranja)` . Cambias el color de la marca en un solo lugar.
- **Selectores**: por clase (`.card-producto`), por id (`#barra`), y combinados. Ahí está el requisito "usar selectores CSS".
- **Responsive con @media** : por ejemplo, en el admin, cuando la pantalla mide 768px o menos, el menú lateral se esconde y aparece la hamburguesa:

```
@media (max-width: 768px) {
  .sidebar { transform: translateX(-100%); } /* escondido */
  body.sidebar-abierto .sidebar { transform: translateX(0); } /* visible */
}
```

Bootstrap ya es responsive por su sistema de **grilla** (`row` y `col-md-4` , `col-lg-3` , etc.): las columnas se reacomodan solas según el ancho. Prueba encogiendo la ventana del navegador.

6. Los datos (arreglos de JavaScript)

Antes de mostrar nada, hay que **tener** los datos. Están en tres archivos:

- `datos-productos.js` → `const PRODUCTOS = [ {...}, {...} ]` . Cada producto es un **objeto** con `codigo` , `nombre` , `precio` , `stock` , etc. La evaluación pide exactamente esto: "crear un arreglo de productos".
- `datos-usuarios.js` → `const USUARIOS = [...]` . Usuarios de prueba para el login (con su `rol`).

- `datos-regiones.js` → `const REGIONES = [...]`. Regiones con sus comunas.

Estos archivos **solo definen** los datos; no dibujan nada. Al cargarse primero, dejan esas variables disponibles para el resto.

7. Mostrar los productos (`tienda.js`)

El requisito dice: "mostrar los productos del arreglo mediante JavaScript". Así se hace:

1. Se define una función `tarjetaProductoHTML(p)` que, dado un producto, devuelve el **texto HTML** de su tarjeta (imagen, nombre, precio y botón "Añadir").
2. Se recorre el arreglo con `.map()` (transforma cada producto en su HTML) y `.join("")` (une todo en un solo texto).
3. Se inyecta ese texto en el contenedor: `contenedor.innerHTML = ...`.

En **productos.html** además hay **buscador** y **filtro por categoría**. Cada vez que escribes o cambias el filtro, se ejecuta `aplicarFiltros()`, que usa `.filter()` para quedarse solo con los productos que coinciden, y vuelve a pintar. Eso es la "búsqueda en tiempo real".

En el **home**, se muestran solo los primeros 8 con `PRODUCTOS.slice(0, 8)`.

Conexión clave: el botón "Añadir" llama a `anadirYAvisar('MC001')`, que a su vez llama a `agregarAlCarrito` (que vive en `carrito.js`). Por eso `carrito.js` debe cargarse antes.

8. El detalle del producto (`detalle.js`)

¿Cómo sabe la página de detalle **cuál** producto mostrar? Por la **URL**.

Cuando haces clic en un producto, vas a `detalle-producto.html?codigo=MC001`. Esa parte `?codigo=MC001` se llama *query string*. El JS la lee así:

```
const params = new URLSearchParams(window.location.search);
const codigo = params.get("codigo");      // "MC001"
const p = buscarProducto(codigo);        // busca ese producto en PRODUCTOS
```

Luego arma el HTML del detalle con su imagen, precio, stock, un selector de cantidad y el botón "Añadir al carrito". También muestra "productos relacionados" de la misma categoría.

9. El carrito con `localStorage` (`carrito.js`)

Esta es una de las partes que más valora la evaluación. `localStorage` es una "cajita" del navegador que **guarda texto y no se borra al recargar**. Como solo guarda texto, convertimos:

- Al **guardar**: objeto → texto con `JSON.stringify`.
- Al **leer**: texto → objeto con `JSON.parse`.

Guardamos el carrito como una lista liviana `[{ codigo, cantidad }]`. Los detalles (nombre, precio) los buscamos en `PRODUCTOS` cuando hace falta.

Funciones principales (todas guardan al final con `guardarCarrito`, que también refresca el contador de la navbar):

- `agregarAlCarrito(codigo)` : si ya está, sube la cantidad; si no, lo agrega.
- `cambiarCantidad(codigo, n)` y `eliminarDelCarrito(codigo)`.
- `calcularSubtotal()` : suma precio × cantidad de todo.
- `aplicarCupon()` : si el código está en `CUPONES`, aplica el descuento.

La página **carrito.html** se dibuja con `renderizarCarrito()`, que crea una fila por producto y calcula subtotal, descuento y total.

Para probar que persiste: agrega productos, recarga la página (F5) y verás que siguen ahí. Eso es `localStorage` funcionando.

10. Validación de formularios (`validaciones.js`)

El corazón del requisito de JavaScript. La idea general:

- Cada campo tiene una función que devuelve `true` / `false` y pinta el estado usando las clases de Bootstrap: `is-invalid` (rojo) o `is-valid` (verde), con el mensaje en un `<div class="invalid-feedback">`.
- Se valida **en tiempo real** (evento `input` / `change`) y también **al enviar** (evento `submit`).
- Al enviar, usamos `e.preventDefault()` para que la página **no** se recargue, y solo si TODO está bien mostramos el mensaje de éxito.

Reglas implementadas (tal cual el enunciado):

- **Login**: correo requerido, máx 100, solo dominios `@duoc.cl`, `@profesor.duoc.cl`, `@gmail.com`; contraseña entre 4 y 10.

- **Registro / Usuario:** RUN válido, nombre (máx 50), apellidos (máx 100), correo (dominios), región y comuna, dirección (máx 300). En el admin además "tipo de usuario".
- **Contacto:** nombre (máx 100), correo (dominios), comentario (máx 500).
- **Producto (admin):** código (texto, mín 3), nombre (máx 100), descripción opcional (máx 500), precio (mín 0, decimales; 0 = gratis), stock (entero ≥ 0), stock crítico opcional, categoría requerida.

10.1 El RUN chileno (dígito verificador)

El enunciado pide "validar si el RUN está correcto". No basta con contar caracteres: hay que calcular el **dígito verificador** con el algoritmo **módulo 11**:

1. Se separa el cuerpo (números) del último carácter (el dígito verificador, DV).
2. Se recorre el cuerpo de derecha a izquierda multiplicando por 2,3,4,5,6,7 y volviendo a 2.
3. Se suma todo, se calcula `11 - (suma % 11)`.
4. Si da 11 → DV es "0"; si da 10 → "K"; si no → el número.
5. Se compara con el DV que escribió el usuario.

Ojo: para probar usa RUTs **válidos** sin puntos ni guion, por ejemplo `123456785`. El "19011022K" del enunciado es solo un ejemplo de **formato**; su DV real no es K, así que el validador (correctamente) lo rechaza.

10.2 Región y comuna dependientes

La función `poblarRegionesComunas()` llena el `<select>` de regiones desde el arreglo `REGIONES`. Cuando cambias la región (evento `change`), vacía y vuelve a llenar el `<select>` de comunas solo con las de esa región. Eso cumple "al cambiar la región, cambian las comunas".

11. Login por roles y protección del admin

- En `login.html`, al enviar, `validaciones.js` busca el correo+contraseña en `USUARIOS`. Si calza, guarda la "sesión" en `localStorage` (`guardarSesion`) y redirige: admin/vendedor → `admin/index.html`; cliente → `index.html`.
- En cada página del admin, `admin.js` revisa al cargar si hay sesión válida. Si no, te devuelve al login (`window.location.href = "../login.html"`). Así las vistas del admin quedan "protegidas".
- **Roles:** si el rol es "vendedor", se ocultan los elementos con clase `solo-admin` (por ejemplo, el menú "Usuarios"). El administrador ve todo.

12. El panel de administrador (`admin.js`)

- **Sidebar** (menú vertical) definido en `admin.css` . En el celular se abre/cierra con la hamburguesa (agrega/quita la clase `sidebar-abierto` en el `body`).
 - **Listados**: las tablas de productos y usuarios se llenan recorriendo `PRODUCTOS` / `USUARIOS` . Los productos en stock crítico muestran una etiqueta roja.
 - **Editar**: los botones de editar llevan a `producto-editar.html?codigo=...` . Ahí `admin.js` lee el código de la URL y **precarga** los datos en el formulario.
 - **Guardar**: por ahora solo valida y muestra "guardado" (no hay base de datos todavía).
-

13. Checklist de la rúbrica (para revisar antes de entregar)

- [x] HTML actual y semántico (secciones, encabezados, párrafos, listas).
 - [x] Navegación: hipervínculos, imágenes/íconos, botones, menús y barra lateral.
 - [x] Formularios para ingresar datos (registro, login, contacto, producto, usuario).
 - [x] Hoja de estilos **externa** y responsive; selectores y propiedades CSS.
 - [x] Validación de formularios con JS, mensajes de error y sugerencias, en tiempo real.
 - [x] Repositorio en GitHub con commits claros (ver sección 14).
-

14. Subir a GitHub y publicar en GitHub Pages (desde cero)

14.1 Crear el repositorio

1. Entra a <https://github.com> e inicia sesión.
2. Botón **New** (o el "+" arriba a la derecha → *New repository*).
3. Nombre: por ejemplo `ferreteria-los-maestros` . Márcalo **Public**. **No** marques "Add a README" (ya tienes uno). Crea el repo.

14.2 Subir los archivos (con Git en tu compu)

Abre una terminal **dentro** de la carpeta del proyecto y ejecuta:

```
git init
git add .
git commit -m "Entrega 1: frontend tienda Ferretería Los Maestros"
git branch -M main
```

```
git remote add origin https://github.com/TU-USUARIO/ferreteria-los-maestros.git
git push -u origin main
```

Reemplaza `TU-USUARIO` por tu usuario de GitHub.

Si prefieres sin terminal: en la página del repo usa **Add file → Upload files**, arrastra todo y confirma. Pero para la rúbrica ("commits claros") es mejor usar Git de verdad.

14.3 Activar GitHub Pages

1. En el repo, ve a **Settings → Pages** (menú izquierdo).
2. En *Build and deployment*, "Source": elige **Deploy from a branch**.
3. Branch: **main**, carpeta / **(root)**. Guarda (**Save**).
4. Espera ~1 minuto y recarga. Aparecerá la URL, algo como: `https://TU-USUARIO.github.io/ferreteria-los-maestros/`
5. Ábrela: ¡tu sitio está publicado! Comparte ese enlace en la entrega.

El archivo `.nojekyll` que viene incluido evita que GitHub Pages ignore ciertos archivos. Déjalo ahí.

14.4 Trabajar en equipo

- El dueño del repo agrega a los compañeros en **Settings → Collaborators**.
- Cada quien clona con `git clone <url>` y trabaja.
- Flujo básico para subir cambios:

```
git pull                # traer lo último del equipo
# ...editas archivos...
git add .
git commit -m "Agrega validación del formulario de contacto"
git push
```

- **Commits claros** (lo pide la rúbrica): mensajes cortos que digan *qué* cambiaste, por ejemplo "Agrega filtro por categoría en productos" en vez de "cambios".
-

15. Preguntas típicas para la presentación (defensa)

Prepárate para responder cosas como:

- *¿Por qué el CSS es externo?* Para mantener el HTML limpio y reutilizar el mismo diseño en todas las páginas (un solo archivo que cambia todo).
- *¿Cómo guardas el carrito?* En `localStorage`, convirtiendo el arreglo a texto con `JSON.stringify`.
- *¿Cómo validas el RUN?* Con el algoritmo módulo 11 del dígito verificador.
- *¿Cómo se muestran los productos?* Desde un arreglo de objetos, recorriéndolo con `.map()` e inyectando el HTML con `innerHTML`.
- *¿Cómo cambian las comunas al elegir región?* Con un evento `change` que vuelve a llenar el `<select>` de comunas.
- *¿Cómo proteges el admin?* Revisando la sesión guardada; si no hay, redirijo al login.

¡Éxito en la presentación! 💪